MDFS-Net: Multidomain Few Shot Classification for Hyperspectral Images With Support Set Reconstruction

Ankit Jha and Biplab Banerjee, *Senior Member, IEEE*

***Abstract*— Deep neural networks are highly specialized for a given task and visual domain, which can limit their practical use. To address this issue, recent studies have proposed to learn universal feature extractors that can be used across multiple domains simultaneously, inspired by the success of transfer learning (TL). However, these universal features are still inferior to specialized networks. In the context of hyperspectral image (HSI) classification, the lack of labeled training samples due to high cost and the restriction to single-domain learning further complicates the problem. To overcome these challenges, we propose a solution that combines the problems of multidomain learning (MDL) and few-shot learning (FSL) for HSI classification. Our goal is to train a highly shareable network with all domains in the low-shot training regime. We call our network the multidomain few-shot network (MDFS-Net), which shares the majority of model parameters (specifically, convolution and dense layer parameters) across domains while keeping domain-specific batch-normalization layers separate to capture domain characteristics. To address the overfitting issue in few-shot models, we supplement the main classification task with an auxiliary self-supervised task. We test our proposed method on five benchmark HSI datasets and find that MDFS-Net consistently outperforms relevant baselines convincingly. Our approach offers a promising solution for HSI classification in remote sensing (RS) by enabling the design of a unified classification system that can work with multiple HSI sites (domains) and fewer labeled samples.**

***Index Terms*— 3-D CNNs, few-shot learning (FSL), hyperspectral image (HSI) classification, multidomain learning (MDL), remote sensing (RS).**

I. Introduction

THE hyperspectral image (HSI) classification task deals with the assignment of the semantic land-cover classes corresponding to the image pixels and traditionally solved using *ad hoc* machine learning approaches [1], [2], [3], [4]. Recently, deep learning methods [5], [6], [7], [8], [9], [10], [11], [12], [13], [14] have subsequently been found to produce superlative performance, given their data-driven spectral-spatial feature learning capabilities. Chen et al. [15] extracts spatial information from HSIs using morphological profiling and constrained energy minimization. Hong et al. [16] superpixel-based subspace learning using multimodal RS data for classification. Nonetheless, the existing HSI classification models largely overlook two important aspects: 1) inference under the availability of fewer training examples and 2) inference under the notion of distribution shifts between the training and test data, respectively. We discuss both issues in detail below.

Few-shot learning (FSL) [17], [18], [19] aims to train a model that can generalize to new classes with limited labeled samples, based on a set of base classes with ample supervision. Meta-learning-based approaches [19], [20] are among the popular FSL methods that use an episodic training to learn generalizable feature extractors. The FSL problem is well-suited for HSI classification, as obtaining labeled data for rare classes can be challenging. Recently, there has been a surge in FSL techniques [21], [22], [23], [24], [25], [26] for HSI. However, existing FSL models are limited to single-domain learning, which makes it difficult to scale up as new models need to be trained for every HSI site. To address this, we need a more scalable approach for HSI classification that can work with multiple sites.

Supervised learning models face a serious problem of being unable to generalize to test samples with different data distributions than the training data [27]. This is a significant issue in remote sensing (RS) since models trained on one geographical area must often be evaluated in other locations. Factors like atmospheric conditions and changes in ground patterns can affect the spectral properties of these images, altering the appearance of the training and test data. Although domain adaptation (DA) [28], [29], [30] or generalization (DG) [31], [32], [33] techniques can handle domain shifts, they are limited to specialized setups. Moreover, these models focus only on the performance of the test domains, which can reduce the performance of the training domains. Ideally, models should provide equivalent performance on data from both source and target domains. This motivates us to address the research question: How can we design a learning model that works with multiple HSI domains without compromising performance on individual domains, especially when the domains have an extremely small set of training samples?

We propose to tackle the problem of learning from multiple HSI domains with the concept of *multidomain learning* (MDL). Our goal is to ensure that the learned domains are not forgotten and that the model can learn from multiple domains without compromising its performance on individual domains. In MDL [34], [35], [36], we strive to develop a feature extractor that is highly shareable across domains

Manuscript received 11 March 2023; revised 15 June 2023 and 13 September 2023; accepted 30 September 2023. Date of publication 16 October 2023; date of current version 2 November 2023. *(Corresponding author: Ankit Jha.)*

Ankit Jha is with the Center of Studies in Resources Engineering, IIT Bombay, Mumbai 400076, India (e-mail: ankitjha16@gmail.com).

Biplab Banerjee is with the Center of Studies in Resources Engineering (CSRE) and the Center for Machine Intelligence and Data Science (CMInDS), IIT Bombay (IITB), Mumbai 400076, India (e-mail: getbiplab@gmail.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/TGRS.2023.3324947, provided by the authors.

Digital Object Identifier 10.1109/TGRS.2023.3324947

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

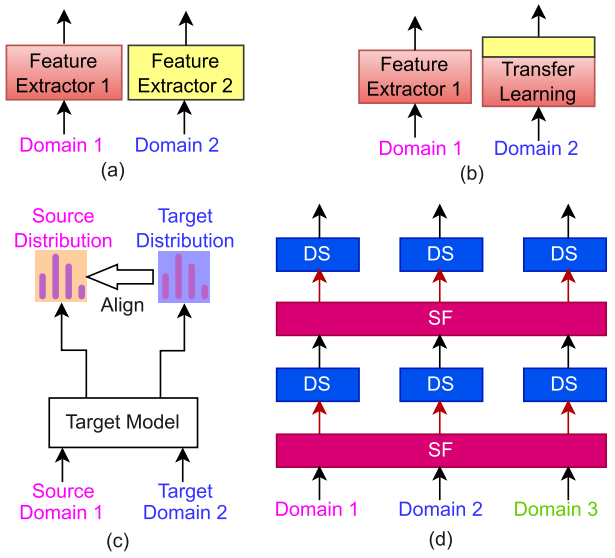


Fig. 1. Comparison between different learning schemes. (a) Individual feature extractor for each domain. (b) TL-based classification. (c) DA between source and target domains, i.e., domain 1 and domain 2. (d) MDL, where SF and DS are features from shared layers and domain-specific layers, respectively. Shared layers consist of a large set of parameters, whereas domain-specific layers are an additional small set of parameters associated with each domain. Best viewed in color.

while also incorporating domain-specific classification modules. This is in contrast to single-domain learning, where domain-specific models are mutually exclusive. MDL differs from related tasks such as DA and transfer learning (TL) because it focuses on learning shared and domain-specific features in a single network, making it more scalable. Fig. 1 clarifies the differences between MDL and DA/TL. To the best of our knowledge, the concept of MDL is new to the RS community and has the potential to solve the problem of learning from multiple HSI sites, each with unique spectral-spatial characteristics, in a compact model. Our ultimate goal is to equip the MDL model with the ability to learn from a minimal set of training samples despite the challenges of overfitting in low-data regimes. Feature extraction in HSI requires a careful selection of spectral and spatial features, and noise bands can interfere with the process. Learning features from multiple domains in the proposed setup, which aims to disentangle domain-specific and shared features, presents a significant challenge. Nevertheless, the problem definition of few-shot MDL (FS-MDL) is generalizable and can be easily adapted to other modalities with minor architectural modifications.

Our Proposed MDFS-Net: Based on the aforementioned considerations, we introduce the setting of FS-MDL for HSI classification in this article. Our goal is to learn an MDL model that can share most of the model parameters among domains while keeping a small set of domain-dependent parameters and avoids overfitting despite having very few training samples per class per domain. To this end, we propose a novel convolutional neural networks (CNN)-based multidomain few-shot network (MDFS-Net) to combat the FS-MDL setting. MDFS-Net is based on the hypothesis that while all domains share the parameters of the feature backbone, we introduce domain-specific batch-normalization layers for capturing the domain-specific feature properties. We implement a prototypes-based nearest-neighbor nonparametric classifier on top of the feature extractor, which is domain-agnostic in nature. As a result, the domains follow different paths in the feature extractor cooperatively, aiding each other in judiciously sharing cross-domain knowledge. To deal with the overfitting issue effectively and ensure better generalization from small training sets, we introduce a combination of an uncertainty optimization loss, which brings regularity in the feature space, and a self-supervised objective that complements the main classification task. The model is meta-trained in a domain incremental fashion to ensure that there is no information loss regarding all domains.

Our **major contributions** are summarized as follows.

1) We propose the concept of FS-MDL within the realm of HSI classification and present an innovative solution, MDFS-Net, to address this challenge.
2) We present an intuitive meta-learning strategy for training model parameters, which are predominantly shared across domains. To retain domain-specific information, we employ separate batch-normalization parameters for each domain. In addressing overfitting, we introduce the concept of minimizing feature uncertainty and incorporate a self-supervised task for sample reconstruction.
3) We conducted experiments on five benchmark HSI datasets, treating each as a distinct domain, and introduced training and evaluation protocols for the FS-MDL task. Our MDFS-Net, as proposed, surpasses all related baselines by a minimum margin of 7% in mean overall accuracy (OA).

II. Related Works

In this section, we discuss the related works from the literature for supervised HSI classification, few-shot HSI classification, and MDL, respectively.

A. Land-Use/Land-Cover Classification From HSI Data

Land-cover classification of HSIs has been approached using traditional [2], [37] and deep learning [7], [8], [10], [38] techniques. To address the noisy and irrelevant bands, dimensionality reduction techniques such as [39], [40], and [41] are commonly used in the preprocessing stage. Conventional methods such as support vector machine (SVM), random forest, and k-nearest neighbors (KNN) only consider hand-crafted spectral features, while others incorporate kernel tricks to enhance the spatial features along with the spectral ones, leading to a significant performance boost [42]. On the other hand, deep learning models based on joint 1-D, 2-D, and 3-D convolution [5], [6], [8], [11], [12], [13], [43], [44] disentangle spectral and spatial feature learning in a data-driven manner, resulting in improved accuracy compared to *ad hoc* techniques. However, they are limited in assessing channel correlations, which are crucial in HSI. To mitigate the effects of noisy bands, spectral-spatial attention learning mechanisms [45], [46], [47] have been integrated into the CNN or Transformer [48], [49], [50] backbones. These attention mechanisms

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

assign weights to the features, ignoring task-irrelevant artifacts. Later, self-supervised learning based methods [51], [52], [53], [54], [55] have been introduced to effectively classify the hyperspectral RS images. However, it is important to note that these models are specialized in single-domain learning and cannot handle multiple HSI domains to train a common model.

B. Few-Shot HSI Classification

Limited labeled data has always been a persistent challenge in RS training. To address this, various methods have been proposed, including semisupervised CNN, self-taught learning, and FSL [56], [57]. FSL tackles the problem through the "learning to learn" paradigm, where a model is trained on a set of fully-supervised base classes to TL to novel, label-deficient categories [18], [19], [58]. Meta-learning algorithms, such as model-agnostic meta-learning (MAML) [18], create few-shot tasks (episodes) to train the model, enabling it to learn invariant features. Metric-learning methods [57] aim to learn an optimal feature space with minimal intraclass and maximum interclass differences.

In this respect, siamese networks [59] compute the similarity between input image pairs, and prototypical networks [17] compute query distance from prototypes to assign labels. Deep FSL [60] uses a metric learning approach to classify few-shot HSI samples. In another work, the relation module between support and query embeddings is induced to compute relational scores, and a statistical extreme value theory-based multitask few-shot HSI classification method called MDL4OW [23] has been proposed. The linear programming incremental learning classifier (LPILC) [21] classifies new classes of HSI datasets incrementally, while soft pseudo-labels are used to avoid overfitting in few-shot HSI classification. Other methods on few-shot HSI classification have been introduced in [25], [60], [61], [62], and [26]. The stable prototypical network (SPN) [24] uses a Bayesian approximation-based regularizer to reduce uncertainties in model predictions. However, these models are unable to handle domain differences effectively, making them unable to solve FS-MDL.

C. Multidomain Learning

The traditional approach to solving classification problems involves training domain-specific inference models from available data distributions. Our goal, however, is to learn multiple data distributions simultaneously. The following briefly discusses the existing solutions to deal with multiple data domains.

As mentioned earlier, the traditional methods to handle multiple domains are through DA/TL and incremental learning (IL). For example, in TL, naive CNN-based pretrained networks are fine-tuned for new tasks. However, learning tasks from scratch can affect computational efficiency and cause the model to forget previously learned features. IL [21], [63], [64] seeks to incrementally learn a variety of tasks while adjusting network parameters or enhancing model capacity, but with the constraint that previous data will not be accessible in future time stamps. Although such models are good at optimal parameter sharing, they cannot explicitly handle domain mismatches among tasks. In DA [28], [31], [65], [66], [67], discrepancies between source and target domains are minimized so that the model learned on the source dataset can be tested over the target dataset by tuning the trained model. Ramirez et al. [68] extend DA to cross-task scenarios where the model transfers learned concepts across different training processes. Recent endeavors aim to amalgamate DA and IL within a unified framework [69]; however, these models do not yet support the FSL setup.

In contrast, MDL trains all domains together using the same backbone. Previously, MDL has been used in computer vision for object classification. For instance, multidomain architectures have been proposed in [34], [70], and [71] for performing image classification tasks across a wide range of visual domains. The majority of parameters are shared across domains, with only a small set of domain-specific parameters [72]. Differently, Rebuffi et al. [34] proposed a new parameterization of the traditional residual network architecture that allows for high parameter sharing between domains while only increasing model parameters by a small percentage. Rebuffi et al. [73] explicitly deals with domain-specific parameters, with a universal parameter shared across all domains and domain-specific modules trained on top of universally learned weights. Finally, early-exit modules were proposed in [71] for the trade-off between domain complexity and performance. To the best of our knowledge, these methods are supervised in nature and may not accommodate a small set of training data.

III. Proposed Method

A. Problem Formulation and Notations

Initially, we define the FSL and MDL problems separately before discussing the FS-MDL setup in detail.

The FSL classification task involves two stages: meta-training and meta-testing. The classes available during meta-training are called base (seen) classes. The trained FSL model is deployed to classify the novel (unseen) class samples at the meta-testing time. During meta-training, we randomly divide the base class samples into support training ($\mathcal{S}$) set and query validation ($\mathcal{Q}$) set in every training episode to simulate the potential testing scenario. This setup is popularly known as the $\mathcal{N}$-way $\mathcal{K}$-shot learning. To be precise, we randomly select $\mathcal{N}$ classes with $\mathcal{K}$ training samples per class to define $\mathcal{S}$. The remaining data constitute $\mathcal{Q}$. Let us define $\mathcal{S} = \{x_i, y_i\}_{i=1}^{\mathcal{N}\mathcal{K}}$ and $\mathcal{Q} = \{x_j\}_{j=1}^{\mathcal{N}\mathcal{M}}$, where $(x, y) \subset \mathcal{X} \times \mathcal{Y}$. Here, $\mathcal{X} \in \mathbb{R}^{H\times W\times B}$, and H, W, and B define the height, width, and depth for the input hyperspectral patches. During meta-testing, we fine-tune the trained model on the support set of the novel classes and evaluate it on the respective unlabeled query set samples.

In MDL, the input data domains have different spectral/spatial distributions. For example, let $(\mathcal{X}_i, \mathcal{Y}_i)$ and $(\mathcal{X}_j, \mathcal{Y}_j)$ be the samples from two different distributions with $P(\mathcal{X}_i, \mathcal{Y}_i)$ and $P(\mathcal{X}_j, \mathcal{Y}_j)$ denoting their distributions, respectively. MDL assumes that $P(\mathcal{X}_i, \mathcal{Y}_i) \neq P(\mathcal{X}_j, \mathcal{Y}_j)$. This assumption implies that both domains are significantly different as per the marginal

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

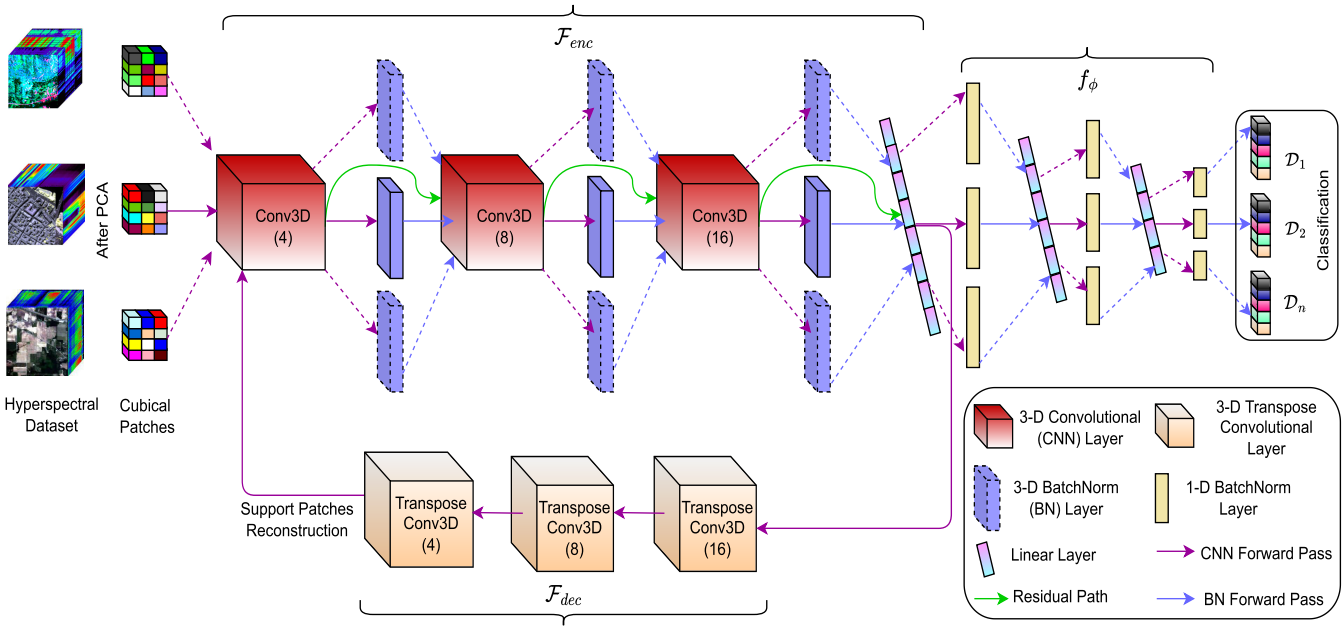


Fig. 2. Block diagram for MDFS HSI classification. BN 3-D and BN 1-D are domain-specific layers, respectively. The support patch features are separated after the flattened layer (latent features) using decoder path $\mathcal{F}_{\text{dec}}$ and used for reconstruction of the episode's support patch samples. Best viewed in color.

and class-conditional distributions. However, they may share some common spectral or spatial characteristics. We can represent this scenario using the sequence $\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_d\}$, where $\mathcal{D}_i = (\mathcal{X}_{\mathcal{D}_i}, \mathcal{Y}_{\mathcal{D}_i})_{i=1}^{d}$. The goal is to train a shared model for all the d domains simultaneously, which is then used for inference over the in-domain data.

Our proposed FS-MDL setup combines FSL and MDL in a meaningful manner. We assume the access to d HSI data domains, where the ith domain has B_i spectral channels. In each domain, the base and novel classes are defined as follows: $\mathcal{D}i^{\text{Base}}$ with labels $\mathcal{Y}_i^{\text{Base}}$, and $\mathcal{D}_i^{\text{Novel}}$ with labels $\mathcal{Y}_i^{\text{Novel}}$, respectively. $(\mathcal{D}_i^{\text{Base}}, \mathcal{D}i^{\text{Novel}}) \subset \mathcal{D}_i$, $i \in \{1, \ldots, d\}$. Note that there are no constraints on the possible overlap of land-cover classes in different domains. Our goal is to meta-train a common feature extractor over all the domains on the respective $\{\mathcal{D}_i^{\text{Base}}\}_{i=1}^{d}$ using episodic training. We randomly sample $(\mathcal{S}, \mathcal{Q})$ from different domains considered sequentially. During meta-testing, the trained model is fine-tuned on the support set of $\{\mathcal{D}_i^{\text{Novel}}\}_{i=1}^{d}$ and evaluated on the query set consisting of data from all the d domains.

B. Data Preprocessing and MDFS-Net Architecture

In this section, we present an overview of the MDFS-Net architecture, which is illustrated in Fig. 2. We begin by considering an input cubical HSI patch $x \in \mathcal{X}$ with dimensions $\mathbb{R}^{H \times W \times B}$ and its corresponding label $y \in \mathcal{Y}$. Note that the description applies to all domains, so we remove the subscript. To ensure consistency in the channel dimensions, we preprocess the HSI data using principal component analysis (PCA) [41]. Specifically, we perform PCA separately for each domain, reducing the dimensionality of the data. We select the top b bands ($b \leq \min\{B_i\}_{i=1}^{d}$) for all domains that preserve at least 95% of the data variance.

The proposed MDFS-Net backbone, denoted by $\mathcal{F}_{\text{enc}}$, is composed of domain-shared $\theta_{\text{sh}}^{\text{enc}}$ and domain-specific $\theta_{\text{sp}}^{\text{enc}}$ parameters. The total parameters in the encoder are given by $\theta^{\text{enc}} = \theta_{\text{sh}}^{\text{enc}} + \theta_{\text{sp}}^{\text{enc}}$. To extract spatial-spectral features across all domains, we use three 3-D CNN layers as the domain-shared layers. However, we introduce domain-specific 3-D batch normalization (BN) after the shared convolution layers to capture domain-specific characteristics that cannot be properly extracted through the shared features. To strengthen the latent feature space and avoid overfitting, we employ a decoder, denoted by $\mathcal{F}_{\text{dec}}$, on top of the encoder subnetwork. The decoder consists of transposed 3-D convolutional layers, which are shared across domains as shown in Fig. 2. The decoder is responsible for reconstructing the support set samples.

To generate class-prototype vectors, we flatten the 3-D feature maps from the final convolution layer of $\mathcal{F}_{\text{enc}}$ and average the embeddings of the support samples per class in each episode. These prototype vectors are then passed through a domain-specific 1-D batch normalization (BN) layer, followed by two fully connected (FC) layers. The classification task of MDFS-Net is based on the concept of prototypes-based metric learning. This idea suggests that a given sample should be close to its same-class prototype while being far from all the other prototypes. To achieve this, we define the MDFS-Net classification head as f_ϕ, where the total parameter space ϕ consists of domain-agnostic parameters ϕ_{sh} and domain-dependent parameters ϕ_{sp}. Rectified linear unit (ReLU) nonlinearity is applied after each layer of the classification head.

C. Domain-Specific Batch Normalization

Since the batch-norm features play an important role in MDFS-Net, we seek to separately discuss the same in this section. The batch normalization layers capture the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

domain-specific feature statistics into their parameters. For the given mini-batch at the lth layer with batch-size N_l with estimated mean μ and variance σ of the activations $x_l \in \mathbb{R}^{H_l \times W_l \times b_l}$ over the batch, we define the batch normalization as follows:

$$\mathrm{BN}(x_l; \gamma, \beta) = \gamma \cdot \hat{x}_l + \beta \tag{1}$$

where x_l is the output from $(l-1)$th layer and can come from any of the d domains, $\hat{x}_l = (x_l - \mu/(\sigma^2 + \epsilon)^{1/2})$. ϵ is a small constant used to avoid divide-by-zero error, and the affine parameters are γ, and β are learnable. We use l to denote the BN layers in Section III-D.

As different domains comprise different characteristics, simply using a common global BN layer for all the domains leads to unstable training and poor generalization since the model, in this case, tends to forget the characteristics of most domains. Hence, this is not suitable while multiple domains are in training. As a remedy, we propose to deploy disjoint domain-specific BN layers with separate affine parameters $\gamma_{\mathcal{D}_i}$ and $\beta_{\mathcal{D}_i}$ for a given $\mathcal{D}_i$. Hence, a given BN layer now consists of $2d$ affine parameters. For a given mini-batch, the domain-specific mean $\mu_{\mathcal{D}_i}$ and standard deviation $\sigma_{\mathcal{D}_i}$ are calculated using the samples from $\mathcal{D}_i$ only. Subsequently, (1) is replaced as follows for domain $\mathcal{D}_i$ at the lth layer to normalize a feature map $x_l \in \mathcal{D}_i$:

$$\mathrm{BN}_{\mathcal{D}_i}(x_l; \gamma_{\mathcal{D}_i}, \beta_{\mathcal{D}_i}) = \gamma_{\mathcal{D}_i} \cdot \hat{x}_l + \beta_{\mathcal{D}_i}. \tag{2}$$

As illustrated in Fig. 2, we employ domain-specific BN3D layers between two consecutive domain-shared layers in $\mathcal{F}$enc, and domain-specific BN1D layers in f. The BN3D layers capture information from each domain with dimensions of $\mathbb{R}^{H_{\mathrm{cb}} \times W_{\mathrm{cb}} \times C_{\mathrm{cb}}}$, while the BN1D layers extract domain-specific information in 1-D space, i.e., $\mathbb{R}^{1 \times \mathrm{dim}_{\mathrm{lin}}}$. Here, H_{cb}, W_{cb}, and C_{cb} denote the height, width, and channel of the previous convolution blocks, and $\mathrm{dim}_{\mathrm{lin}}$ represents the linear dimension from the respective linear layers. Furthermore, we utilize dropout layers together with the BN layers to provide regularization effects in MDFS-Net.

D. Training Paradigm of MDFS-Net in Detail

In this section, we will elaborate on the training protocol used in MDFS-Net, which is performed episodically. Our goal is to obtain a domain-independent $\mathcal{F}_{\mathrm{enc}} + f$ given $\{\mathcal{D}_i^{\mathrm{Base}}\}_{i=1}^{d}$. In each episode, we follow a sequential training strategy for the domains. First, we randomly order the domains and train the model using the support sets from the aforementioned domains. For domain $\mathcal{D}_i$, we begin by constructing episodes with randomly sampled $\mathcal{N}$ classes from $\mathcal{D}_i^{\mathrm{Base}}$ in both the support ($\mathcal{S}$) and query ($\mathcal{Q}$) sets. We then pass ($\mathcal{S}$, $\mathcal{Q}$) through the feature encoder $\mathcal{F}_{\mathrm{enc}}$, followed by f, to obtain the latent embeddings for the samples in ($\mathcal{S}$, $\mathcal{Q}$). The goal of the few-shot learner is to obtain the class prototypes c_k [see (3)] for the kth class by averaging the classwise support-set embeddings

$$c_k = \frac{1}{|\mathcal{S}_k|} \sum_{(x_i, y_i) \in \mathcal{S}_k} f_\phi(\mathcal{F}_{\mathrm{enc}}(x_i)) \tag{3}$$

where $\{\mathcal{S}_k\}$ denotes the set of examples labeled with class k and $k \in \{1, \mathcal{N}\}$. The learning objectives for the meta-training stage are threefold.

1) Learn a discriminative embedding space through the prototype-based classification loss.
2) Ensure better network generalization through an uncertainty minimization loss.
3) Avoid overfitting through a self-supervised loss (SSL).

They are discussed in the following.

1) Prototypical Loss: Next, we compute the Euclidean distance (D_e) and cosine similarity (C_s), denoted by $\langle \cdot \rangle$, between the support class prototype c_k and query-set samples from $\mathcal{Q}$. The objective is to minimize the distance between the query samples and the correct class prototype while maximizing the distance between the query samples and all negative class prototypes, thereby creating a classwise discriminative embedding space.

It is important to note that the query-set samples during the meta-training act as pseudo-test samples since we already know their class labels.

To obtain the class prediction of the query samples, we apply the softmax function to the negative of the computed distances and similarity scores, as mentioned in (4) and (5), respectively. It has been experimentally found that using both metrics together boosts the performance of FS-MDL

$$\mathcal{L}_{\mathrm{Proto}}^{D_e} = \mathop{\mathbb{E}}_{(x_j, y_j) \in \mathcal{Q}} \frac{\exp^{-D_e(f_\phi(x_j), c_k)}}{\sum_{k'} \exp^{-D_e(f_\phi(x_j), c_{k'})}} \tag{4}$$

$$\mathcal{L}_{\mathrm{Proto}}^{C_s} = \mathop{\mathbb{E}}_{(x_j, y_j) \in \mathcal{Q}} \frac{\exp^{\langle f_\phi(x_j), c_k \rangle}}{\sum_{k'} \exp^{\langle f_\phi(x_j), c_{k'} \rangle}}. \tag{5}$$

2) Uncertainty Optimization Loss to Prevent Overfitting: Typically, we feed input samples through the network only once and calculate the loss. However, the presence of dropout layers in the model may prevent some nodes from learning meaningful features during this one-time feed-forwarding. Instead, we propose leveraging the stochasticity of dropout to learn a more generalizable $\mathcal{F}_{\mathrm{enc}} + f$ in MDFS-Net. Specifically, we introduce the concept of Monte-Carlo (MC) dropout, where we pass the same sample x for $\mathcal{T}$ times through the network in each episode. The dropout layers cause the model to produce different feature embeddings for x at each execution. We define the variance of the feature embeddings obtained for x over the $\mathcal{T}$ runs as the *uncertainty* associated with the network.

To ensure that the network learns generalizable embeddings with reduced uncertainty, we propose minimizing the uncertainty in the feature embeddings. In MDFS-Net, we use (μ, σ) of the domain-specific BN layers to define the uncertainty minimization loss, as these parameters can capture the data distributions in each episode. Specifically, we aim for the model to produce the same (μ_l, σ_l) for the lth BN layer over $\mathcal{T}$ iterations. We achieve this by enforcing the model to reduce the variance Var_l^{μ} for $\mu_l^1, \mu_l^2, \cdots, \mu_l^{\mathcal{T}}$ and Var_l^{σ} for $\sigma_l^1, \sigma_l^2, \cdots, \sigma_l^{\mathcal{T}}$, respectively. We propose carrying out this task for all the BN layers in MDFS-Net.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Steps for MDFS-Net. $\mathcal{S}$ and $\mathcal{Q}$ Denote the Support and Query Set, Respectively. $\mathcal{K}$ Defines the Number of Shots in the Training Set, and $\mathcal{N}$ Is the Number of Classes per Episode. RandomsampleGenerator$(\mathcal{S}, \mathcal{N})$ Randomly Draws Support Set $\mathcal{S}_k$ and Query Set $\mathcal{Q}_k$ From Domain $\mathcal{D}_i$, Where k Is the kth Class and $k \in \{1, \mathcal{N}\}$. x_{i_s} Is the Input Support Patch Samples From the Base Classes. Calculate_Variance Computes Variance for μ_l and σ_l of the lth Batch Normalization (BN) Layer

Input Training domains $\mathcal{D} = \{\mathcal{D}_i\}_{i=1}^{d}$, where $\mathcal{D}_i = \{(x_1^i, y_1^i), \ldots, (x_n^i, y_n^i)\}_{i=1}^{d}$. y represents the ground truth for the input x.
Output The loss $\mathcal{L}_{Total}$ computed for a randomly generated training episode for each domain d.

$Shuffle(\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_d)$ ▷ Randomly shuffling the order for all the domains $\mathcal{D}$
for e in $\{1, \ldots, E\}$ **do** ▷ Select domain from domain $\mathcal{D}_i$ from the shuffled domain $\mathcal{D}$
  for i in $\{1, \ldots, d\}$ **do** ▷ Constructing episodes for E times
    $\mathcal{S}, \mathcal{Q} \leftarrow RandomSampleGenerator(\mathcal{D}_i, (\mathcal{N}, \mathcal{K}))$ ▷ Select support and query samples
    $c_k \leftarrow \frac{1}{|\mathcal{S}_k|} \sum_{(x_i, y_i) \in \mathcal{S}_k} f_\phi(\mathcal{F}_{enc}(x_i))$ ▷ Generating class prototypes from the support samples
    $Arr_\mu = 0,\ Arr_\sigma = 0$ ▷ Initialize the arrays
    for l in the set of BN layers **do** ▷ For each lth BN layer
      $arr_\mu^l = 0,\ arr_\sigma^l = 0$ ▷ Initialize the arrays
      for t in $\{1, \ldots, \mathcal{T}\}$ **do** ▷ Iterating episode for $\mathcal{T}$ times to reduce uncertainty in class-predictions
        Obtain μ_l^t and σ_l^t
        $arr_\mu^l = arr_\mu^l + \mu_l^t$ ▷ Update μ_l for l^{th} BN layer
        $arr_\sigma^l = arr_\sigma^l + \sigma_l^t$ ▷ Update σ_l for l^{th} BN layer
      end for
      $Var_l^\mu = Calculate_Variance(arr_\mu^l)$ ▷ Calculate variance of μ_l for uncertainty minimization
      $Var_l^\sigma = Calculate_Variance(arr_\sigma^l)$ ▷ Calculate variance of σ_l for uncertainty minimization
      $Arr_\mu \leftarrow Arr_\mu + Var_l^\mu,\ Arr_\sigma \leftarrow Arr_\sigma + Var_l^\sigma$ ▷ Update the arrays
    end for
    Optimize $\arg\min_{(\theta^{enc}, \theta^{dec}, \phi)} \mathcal{L}_{Total}$ ▷ Solve $\mathcal{L}_{Total}$ with parameters θ^{enc}, θ^{dec} and ϕ defined in Eq. 8
  end for
end for

Denoting the index l iterating through the BN layers, we present the uncertainty minimization loss as follows:

$$\mathcal{L}_{\text{Var}} = \mathop{\mathbb{E}}_{(x_j, y_j) \in \mathcal{S}} \sum_{l \in \text{BN}} \left[\text{Var}_l^\mu + \text{Var}_l^\sigma\right]. \tag{6}$$

3) Self-Supervised Loss : We also introduce an auxiliary SSL to address the issue of overfitting. Due to the limited number of samples per class used in each episode, the classification loss is susceptible to memorizing the training data. The SSL is designed to penalize such behavior of the network in the few-shot scenario.

To achieve this, we propose to use the reconstruction of the support set samples through the decoder $\mathcal{F}_{\text{dec}}$ as the objective for the SSL. This is defined as follows:

$$\mathcal{L}_{\text{SSL}} = \mathop{\mathbb{E}}_{(x_j, y_j) \in \mathcal{S}} ||x_j - \mathcal{F}_{\text{dec}}(\mathcal{F}_{\text{enc}}(x_j))||_1^1. \tag{7}$$

4) Total Loss: We define the total loss ($\mathcal{L}_{\text{Total}}$) as a combination of (4)–(7) as follows, where α defines the weight decay parameters. We consider the joint optimization strategy where the gradient of $\mathcal{L}_{\text{Total}}$ is calculated with respect to the model parameters for iterative updates

$$\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{Proto}}^{D_e} + \mathcal{L}_{\text{Proto}}^{C_s} + \alpha[\mathcal{L}_{\text{Var}} + \mathcal{L}_{\text{SSL}}]. \tag{8}$$

Similarly, we iterate the above strategies and optimize the MDFS-Net over all the RS domains in $\mathcal{D}_i$.

E. Inference Strategy

During the meta-testing phase of MDFS-Net, we combine the support-set samples over all the d domains and fine-tune MDFS-Net by optimizing $\mathcal{L}_{\text{Total}}$. The model is subsequently used to predict the class labels for the respective query samples by assessing the similarity with the prototype vectors.

IV. Experimental Evaluations

In this section, we discuss the HSIs datasets, experimental protocols, and evaluation metrics used in our proposed work. We use five different RS domains to examine the efficacy of MDFS-Net for HSI classification. The primary goal is to see if the suggested work can perform well in all domains simultaneously.

A. Dataset Descriptions

We perform experiments with six benchmark HSI datasets, namely Indian Pines (IP), Salinas, Botswana, Kennedy Space Center (KSC), University of Pavia (PaviaU), and Houston 2013. These datasets consist of different spectral signatures, spatial resolutions, and numbers of classes. Also, we explain the mentioned dataset is present in the Supplementary Material.

Since the number of spectral bands and image dimensions vary across the aforementioned datasets, some initial preprocessing steps are required to maintain consistency in the multidomain architecture. First, to prevent spectral redundancy

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I

COMPARISON OF MDFS-NET ON HYPERSPECTRAL DATASETS WITH SOTA FEW-SHOT METHODS. THE RESULTS ARE SHOWN ON ONE-SHOT AND FIVE-SHOT SETTINGS FOR FIVE RS HSI DOMAINS. WE HIGHLIGHT THE BEST RESULTS IN **BOLD**. * REPRESENTS RESULTS ON TEN-SHOT SETTING

Method	Dataset															
	1-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**			
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	
Relation [56]	77.46±0.40	69.99±0.28	65.59±0.32	71.84±0.44	65.76±0.39	64.13±0.51	69.73±0.27	68.08±0.42	69.98±0.33	63.34±0.29	55.81±0.40	66.55±0.31	54.65±0.52	53.90±0.38	54.21±0.45	
MAML [18]	74.61±0.36	68.35±0.32	72.13±0.41	72.66±0.33	67.09±0.28	71.93±0.24	69.00±0.37	66.54±0.32	68.42±0.40	61.97±0.35	58.10±0.31	60.76±0.43	56.83±0.38	56.01±0.32	55.96±0.38	
Matching [57]	76.05±0.37	69.73±0.30	65.10±0.35	70.92±0.26	65.08±0.29	63.14±0.21	67.45±0.36	67.92±0.41	68.16±0.33	64.27±0.32	56.48±0.39	65.98±0.46	56.60±0.52	55.86±0.43	56.33±0.41	
Prototypical [17]	78.16±0.29	72.33±0.26	76.59±0.40	73.01±0.35	68.62±0.47	69.86±0.41	68.07±0.27	65.88±0.31	68.32±0.42	67.11±0.34	65.59±0.32	66.27±0.41	67.95±0.33	66.12±0.31	67.46±0.35	
SSFT [62]	77.53±0.33	74.10±0.37	75.66±0.27	72.39±0.39	68.00±0.30	68.71±0.29	68.09±0.35	67.19±0.38	67.50±0.31	66.74±0.28	66.00±0.23	66.49±0.36	67.20±0.30	67.61±0.23	66.54±0.31	
DFS [60]	77.90±0.42	73.58±0.34	76.19±0.33	74.20±0.25	67.51±0.31	71.00±0.24	69.55±0.41	67.28±0.35	68.51±0.38	66.83±0.30	66.35±0.27	66.40±0.32	68.10±0.42	67.53±0.30	68.03±0.29	
BT-Lin [61]	77.51±0.39	75.29±0.42	75.80±0.34	70.56±0.33	68.70±0.39	69.28±0.31	68.19±0.44	66.52±0.41	67.96±0.49	67.38±0.35	66.21±0.40	65.75±0.43	69.60±0.35	66.00±0.41	68.57±0.41	
SPN [24]	78.94±0.21	76.68±0.19	77.05±0.21	76.20±0.26	71.19±0.22	75.33±0.18	72.71±0.20	67.05±0.27	69.93±0.21	68.44±0.26	68.08±0.29	67.59±0.24	71.41±0.20	68.57±0.25	70.99±0.23	
MDFS-Net	**89.23±0.19**	**88.40±0.11**	**88.95±0.28**	**88.50±0.15**	**84.48±0.10**	**87.21±0.29**	**76.45±0.32**	**75.83±0.25**	**78.65±0.17**	**75.79±0.20**	**74.12±0.28**	**75.06±0.11**	**85.80±0.19**	**84.37±0.23**	**84.60±0.13**	
	5-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**			
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	
Relation [56]	80.97±0.37	74.87±0.42	79.76±0.39	75.72±0.33	67.76±0.31	69.88±0.40	75.88±0.31	70.42±0.40	75.71±0.29	72.92±0.25	67.06±0.30	70.59±0.24	72.41±0.32	71.20±0.28	72.54±0.32	
MAML [18]	78.02±0.34	73.30±0.41	77.68±0.53	76.84±0.47	75.44±0.50	76.95±0.41	76.35±0.29	71.19±0.31	73.86±0.43	72.55±0.40	71.31±0.50	71.64±0.42	74.03±0.48	73.66±0.41	74.25±0.50	
Matching [57]	82.15±0.28	75.05±0.35	79.63±0.31	78.11±0.53	76.00±0.46	77.74±0.43	76.33±0.37	71.28±0.40	74.05±0.32	74.16±0.51	73.00±0.43	74.08±0.42	74.95±0.37	73.00±0.34	74.62±0.46	
Prototypical [17]	86.35±0.45	85.21±0.39	85.74±0.36	84.29±0.51	83.18±0.31	83.99±0.35	76.59±0.38	74.90±0.34	74.22±0.38	75.23±0.31	73.89±0.40	73.97±0.48	77.64±0.45	75.19±0.56	77.83±0.51	
SSFT [62]	84.77±0.29	78.70±0.31	82.40±0.32	87.50±0.32	84.22±0.42	84.75±0.31	77.20±0.30	74.02±0.26	75.98±0.31	75.88±0.30	74.61±0.37	73.69±0.32	81.90±0.48	77.53±0.40	80.63±0.39	
DFS [60]	87.43±0.30	85.91±0.38	85.22±0.40	86.11±0.34	84.57±0.31	84.63±0.37	75.00±0.24	73.49±0.42	74.15±0.44	74.72±0.25	73.40±0.29	73.05±0.34	79.46±0.30	78.50±0.37	79.26±0.33	
S3Net[25]	70.26±3.66	66.65±3.94	82.17±2.01	91.62±2.17	90.70±2.38	**95.70±1.19**	-	-	-	-	-	-	79.03±5.94	85.08±3.97	73.44±6.92	
MultiStg[26]*	-	-	-	95.12±0.85	94.57±0.95	97.27±0.53	-	-	-	-	-	-	91.57±3.49	89.11±4.28	93.80±1.39	
BT-Lin [61]	89.15±0.40	83.06±0.34	84.66±0.39	87.50±0.42	84.93±0.46	85.11±0.40	76.82±0.35	72.10±0.38	73.60±0.31	75.00±0.40	74.37±0.45	73.90±0.32	81.20±0.30	76.10±0.42	78.40±0.35	
SPN [24]	90.64±0.18	87.98±0.22	90.63±0.20	89.41±0.27	85.25±0.23	87.93±0.26	77.40±0.21	73.87±0.28	75.65±0.29	76.97±0.32	73.70±0.24	74.55±0.18	83.83±0.25	81.56±0.22	82.17±0.27	
MDFS-Net	**99.89±0.11**	**99.72±0.20**	**99.81±0.29**	**96.34±0.25**	**94.17±0.18**	95.67±0.21	**82.63±0.25**	**79.56±0.20**	**79.40±0.31**	**80.94±0.23**	**80.39±0.18**	**81.13±0.22**	**91.60±0.27**	**90.47±0.23**	**90.35±0.19**	

across all the datasets, we employ PCA with 30 principal components. Second, we generate cubical patches with appropriate zero padding of dimensions (11, 11, c) for each spatial location, where c represents the number of spectral bands selected after performing PCA. We show ablation on choosing the number of PCA components in the Supplementary Material.

B. Implementation and Training Details

For all datasets except Pavia U., we split the data uniformly, selecting six classes for training and testing, with the exception of Pavia U. where we chose five classes for training and four classes for testing. We considered $\mathcal{N} = 5$, $\mathcal{K} = 1, 5$, which represents the number of shots/samples in the support set, and $\mathcal{M} = 15$ for the query set during training.

Our MDFS-Net architecture consists of three 3-D CNN layers that encode feature maps of depth 4, 8, and 16. We use linear layers to generate linear feature embeddings of dimension 128. Finally, to reconstruct the support patches from the latent space, we add three 3-D transposed CNN layers of size 16, 8, and 4. We set the dropout probability to 0.6 for linear layers and use the Adam optimizer [74] with a learning rate of $2e^{-4}$. We found that high α values did not produce good results, so we chose a value of α in the range of [0.75, 1] and set it to 1 for our experiments. As we iterate each episode multiple times, we find that $\mathcal{T} = 5$ best suits our FS-MDL setup. We optimally run all the comparative state-of-the-art (SOTA) methods with the set up mentioned in respective literature. Also for spectral–spatial Siamese network (S3Net) [25] and multistage relation network (MultiStg) [26], we directly compare the results with the available few-shot settings, mentioned in Table I.

During the experiments, the models were trained using a system with Python 3.7 and PyTorch 1.10.1 [75]. The system was equipped with an Intel Xeon[1] CPU E5-2620 processor, 64-GB RAM, and a 12-GB GeForce RTX 3080 Ti GPU card.

[1]Registered trademark.

C. Evaluation Protocols

To evaluate the performance of our proposed MDFS architecture for classifying hyperspectral RS scenes, we consider OA, average accuracy (AA), and kappa coefficient (κ). When testing the MDFS-Net on novel classes, we iterate each domain sequentially in an episodic manner. Specifically, we feed the 3-D HSI patches for each domain $\mathcal{D}_i$ to the trained feature extractor $\mathcal{F}$ to extract the class prototypes, which are then used for classification using the aforementioned metrics.

V. RESULTS AND ANALYSIS

A. Comparison Between MDFS-Net and SOTAs

Table I presents the experimental results for five-way one-shot and five-way five-shot on all the hyperspectral datasets mentioned in Section IV. We compare our proposed method with SOTA methods such as RelationNet [56], MAML [18], Matching [57], Prototypical [17], subspace classifier and feature transformation (SSFT) [62], Barlow-Twins linear (BT-Lin) [61], S3Net [25], MultiStg [26], deep few-shot (DFS) [60], and SPN [24]. We first discuss the OA performance of MDFS-Net on the IP dataset and then on subsequent hyperspectral RS scenes.

For five-way one-shot, we achieve higher OA than the aforementioned SOTAs, i.e., RelationNet [56], MAML [18], Matching [57], Prototypical [17], SSFT [62], BT-Lin [61], DFS [60], and SPN [24] by 13.19%, 16.38%, 14.77%, 12.41%, 13%, 12.7%, and 11.53%, respectively, on the IP dataset. On the other hand, for five-way five-shot setting, our proposed MDFS-Net obtains 18.94%, 21.89%, 17.76%, 13.55%, 15.12% 12.46%, 29.63%, 10.74% and 9.26% higher OA, in comparison with RelationNet [56], MAML [18], Matching [57], Prototypical [17], SSFT [62], DFS [60], BT-Lin [61], S3Net [25], and SPN [24], respectively. Notably, MDFS-Net achieves higher precision on the novel classes of the IP dataset such as woods, hay-windowed, and soyabean-clean compared to the remaining novel classes. On the Salinas dataset, our proposed MDFS-Net reports par performance for the five-way one-shot in compared with RelationNet [56]

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

by 16.66%, MAML [18] by 15.84%, Matching [57] by 17.58%, Prototypical [17] by 15.49%, SSFT [62] by 16.11%, DFS [60] by 14.30%, BT-Lin [61] by 17.94% and SPN [24] by 12.30%. On the other hand, for a five-way five-shot setting, we outperform the referred few-shot RelationNet [56], MAML [18], Matching [57], Prototypical [17], SSFT [62], DFS [60], S3Net [25], BT-Lin [61], and SPN [24], by 20.62%, 19.50%, 18.23%, 12.08%, 8.84%, 10.23%, 4.72%, 8.84%, and 6.93%, respectively. Further, we directly compare the results of MultiStg [26] on Salinas dataset with MDFS-Net under ten-shot setting, denoted as * in Table I.

We exhibit the MDFS-Net findings for five-way one-shot and five-way five-shot settings, similar to IP and Salinas datasets, with encouraging results on the Botswana dataset. We outperform SPN [24] by 4.89% and 6.33% when comparing one-shot to five-shot learning. MDFS-Net's one-shot and five-shot percentage performance differences for the remaining techniques are +8.79 and +8.17 on RelationNet [56], +9.74, and +7.60 on MAML [18], +11.77 and +7.62 on Matching [57], and +10.96, +7.31 on Prototypical network [17], 10.8% and 9% on SSFT [62], 6.6%, and 9.2% on DFS [60], and 8.3%, and 5.8% on BT-Lin [61].

In the multidomain setting for few-shot HSI classification, on the KSC dataset, the MDFS-Net's five-shot classification performance improves the one-shot by a fair margin of 6%. Considering the referred SOTAs few-shot algorithms, we report improvements in classifying novel classes like water, cattail, and broadleaf both in one-shot and five-shot scenarios.

Finally, we evaluate the performance of MDFS-Net on the Pavia U. dataset. Compared to the other hyperspectral domains, the total number of novel classes in Pavia U. is 4, so we chose a four-way few-shot classification for testing. In this article, we compare MDFS-Net with RelationNet [56], MAML [18], Matching [57], Prototypical [17], SSFT [62], DFS [60], BT-Lin [61] and SPN [24], where we outperform the corresponding FSL approaches by 31.15%, 28.97%, 29.20%, 17.85%, 18.60%, 17.70%, 16.20%, and 14.39%, respectively, on four-way one-shot setting. We also report OA of MDFS-Net under four-way five-shot scenario, i.e., MDFS-Net outperforms by 19.19% on, 19.18% on RelationNet [56], 17.57% on MAML [18], 16.65% on Matching [57], 13.96% on Prototypical [17], 10.00% on SSFT [62], 12.14% on DFS [60], 12.57% on S3Net [25], 10.40% on BT-Lin [61], 7.77% on SPN [24], respectively. Also, we directly compare the results of MultiStg [26] on Pavia U. dataset with MDFS-Net under ten-shot setting, denoted as * in Table I.

In Table I, we have also reported the performance of MDFS-Net with other metrics like κ and AA, where we clearly outperform the referred FSL methods on κ and AA at least by 13.30%, and 12.18% in five-way one-shot setting and 9%, and 7.82% in five-way five-shot setting, respectively, considering all the RS HSI domains.

1) Comparison for the Number of Trainable Parameters: In Table II, we quantitatively mentioned the number of trainable parameters present in referred SOTA algorithms and our proposed MDFS-Net. We observe that the MDFS-Net contains more parameters than RelationNet [56], MAML [18], Matching [57], Prototypical [17], DFS [60] and SPN [24] for a single domain. Whereas, with the increase in the number of input RS domains ($\times$ 5), in MDFS-Net, the total number of trainable parameters increases by 1.12×10^3 per domain, which is much lesser than the referred algorithms. Conclusively, MDFS-Net provides better performance with the optimal amount of training parameters.

TABLE II

COMPARISON OF THE NUMBER OF TRAINABLE PARAMETERS IN THE MDFS-NET AND REFERRED SOTAS METHODS. THE NUMBER IN **BOLD** REPRESENTS THE OPTIMAL NUMBER OF PARAMETERS IN THE MDFS SETTING WITH FIVE HSI DOMAINS

Methods	Number of Domains	
	$\times$ 1	$\times$ 5
Relation [56]	1.66×10^6	8.30×10^6
MAML [18]	3.74×10^6	18.70×10^6
Matching [57]	1.66×10^6	8.30×10^6
Prototypical [17]	1.66×10^6	8.30×10^6
DFS [60]	1.60×10^6	8.00×10^6
BT-Lin [61]	1.58×10^6	7.90×10^6
SPN [24]	1.66×10^6	8.30×10^6
MDFS-Net (Ours)	7.60×10^6	**7.61** $\times 10^6$

TABLE III

BASELINE SETTINGS FOR THE MDFS-NET, WHERE D_e, C_s, AND SSL DENOTE EUCLIDEAN DISTANCE, COSINE SIMILARITY, AND RECONSTRUCTION AS THE SSL

Baselines	Settings		
	D_e	C_s	SSL
Baseline-I	✓	✗	✗
Baseline-II	✗	✓	✗
Baseline-III	✓	✗	✓
Baseline-IV	✗	✓	✓
Baseline-V	✓	✓	✗
MDFS-Net	✓	✓	✓

B. Ablation Analysis

1) Baseline Ablations: Table IV shows that we ablate our proposed method with five different baselines (Baselines I–V). We use ED and CS to compute the feature embeddings in baseline-I, and II, respectively, and baseline-II improves the OA for five-way one-shot by 4.16% and five-way five-shot by 1.34% averagely overall, the domains. We further check the performance of multidomain image classification by adding the SSL [defined in (7)] for the support set and referred to as baseline-III and IV. The reconstruction task increases baseline-I and II classification scores by 5.6% and 1.2% on five-way one-shot and 1.9% and 1.1% on five-way five-shot, respectively, across all domains. We also infer that the auxiliary task is more advantageous for one-shot than five-shot. Finally, we define the baseline-V by computing the feature embedding together with ED and CS. This setting aids in generating better class prototypes than the previously stated baselines (i.e., Baselines I–IV), which finally produces optimal MDFS classification performance.

2) MDFS-Net's Performance With Domain Increment: We further evaluate the efficacy of the proposed MDFS-Net by simultaneously increasing the number of domains. From Table V, we observe that learning multiple domains provides better classification scores than the individual ones. As we increase the number of domains, the performance also effectively improves from the previous domain setting.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

WE SHOW THE ARCHITECTURE ABLATION FOR OUR PROPOSED MDFS-NET. THE BASELINE DEFINITIONS ARE PRESENTED IN TABLE III. THE RESULTS ARE SHOWN ON ONE-SHOT AND FIVE-SHOT SETTINGS FOR FIVE RS HSI DOMAINS, DEFINED IN SECTION IV-A. WE HIGHLIGHT THE BEST RESULTS IN **BOLD**

Method	Dataset														
1-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**		
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA
Baseline-I	87.05±0.22	86.77±0.16	87.61±0.19	84.92±0.25	79.80±0.21	83.25±0.32	72.87±0.29	66.52±0.24	70.63±0.35	68.05±0.14	67.16±0.20	65.74±0.11	73.40±0.23	69.42±0.29	72.96±0.22
Baseline-II	88.37±0.17	84.50±0.12	86.98±0.15	85.63±0.11	80.55±0.20	82.19±0.24	72.50±0.23	68.77±0.11	69.95±0.18	74.44±0.16	68.02±0.21	75.18±0.26	82.00±0.19	77.98±0.20	82.15±0.14
Baseline-III	87.49±0.14	85.32±0.10	87.11±0.18	87.15±0.16	83.92±0.13	85.47±0.12	75.60±0.18	74.79±0.20	75.41±0.22	74.25±0.17	71.64±0.11	73.07±0.16	84.67±0.12	84.23±0.19	84.41±0.20
Baseline-IV	88.67±0.23	86.71±0.21	87.99±0.26	87.29±0.19	84.33±0.21	86.11±0.16	75.63±0.22	75.03±0.18	75.38±0.25	74.82±0.20	72.13±0.23	72.98±0.21	84.45±0.16	83.72±0.11	83.69±0.19
Baseline-V	88.89±0.20	88.06±0.15	88.57±0.21	87.9±0.185	85.18±0.20	87.42±0.23	76.14±0.34	75.60±0.31	74.93±0.35	75.36±0.29	73.41±0.24	74.22±0.18	85.00±0.25	85.14±0.14	84.08±0.11
MDFS-Net	**89.23±0.19**	**88.40±0.11**	**88.95±0.28**	**88.50±0.15**	**84.48±0.10**	**87.21±0.29**	**76.45±0.32**	**75.83±0.25**	**78.65±0.17**	**75.79±0.20**	**74.12±0.28**	**75.06±0.11**	**85.80±0.19**	**84.37±0.23**	**84.60±0.13**
5-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**		
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA
Baseline-I	98.90±0.26	98.14±0.24	99.45±0.29	89.37±0.21	85.55±0.25	87.22±0.30	77.95±0.28	73.10±0.19	78.98±0.26	74.50±0.23	71.19±0.31	72.59±0.33	88.80±0.30	84.61±0.35	88.60±0.32
Baseline-II	99.13±0.30	99.48±0.34	99.61±0.29	92.70±0.38	87.54±0.33	91.98±0.31	79.42±0.25	76.11±0.23	78.84±0.29	75.21±0.37	70.86±0.28	74.33±0.30	89.01±0.24	85.33±0.36	88.49±0.27
Baseline-III	98.89±0.18	98.46±0.21	99.13±0.16	92.00±0.26	89.28±0.31	91.74±0.29	78.69±0.23	76.35±0.20	77.98±0.25	75.18±0.18	72.65±0.22	75.06±0.19	90.71±0.26	86.00±0.20	88.83±0.21
Baseline-IV	99.30±0.23	99.27±0.28	99.20±0.31	95.95±0.33	95.79±0.27	95.80±0.22	79.16±0.30	76.82±0.32	78.91±0.27	75.93±0.33	73.28±0.31	76.30±0.25	90.18±0.29	88.39±0.22	90.42±0.20
Baseline-V	99.63±0.36	98.46±0.33	99.35±0.28	95.95±0.22	95.79±0.20	95.82±0.18	79.93±0.35	76.16±0.24	78.27±0.32	78.65±0.20	79.08±0.24	78.99±0.28	90.83±0.33	89.50±0.30	89.99±0.34
MDFS-Net	**99.89±0.11**	**99.72±0.20**	**99.81±0.29**	**96.34±0.25**	**94.17±0.18**	**95.67±0.21**	**82.63±0.25**	**79.56±0.20**	**79.40±0.31**	**80.81±0.23**	**80.39±0.18**	**81.13±0.22**	**91.60±0.27**	**90.47±0.23**	**90.35±0.19**

TABLE V

WE SHOW THE PERFORMANCE OF MDFS-NET WITH THE INCREASE IN THE NUMBER OF DOMAINS. THE RESULTS ARE SHOWN ON ONE-SHOT AND FIVE-SHOT SETTINGS FOR FIVE RS HYPERSPECTRAL DOMAINS, DEFINED IN SECTION IV-A. WE HIGHLIGHT THE BEST RESULTS IN **BOLD**

Domains	Dataset														
1-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**		
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA
1	85.80±0.32	80.09±0.27	85.23±0.30	85.95±0.22	81.40±0.19	83.72±0.29	74.48±0.43	70.66±0.35	74.09±0.39	75.20±0.48	68.34±0.27	73.11±0.34	82.10±0.30	83.34±0.28	82.97±0.30
2	87.50±0.27	84.62±0.25	84.36±0.23	-	-	-	-	-	-	-	-	-	83.00±0.11	82.04±0.26	82.27±0.18
3	88.41±0.28	87.53±0.30	88.33±0.25	83.65±0.32	79.91±0.40	83.15±0.34	-	-	-	-	-	-	83.64±0.21	83.72±0.29	83.49±0.23
4	89.01±0.25	88.46±0.21	88.57±0.26	85.33±0.30	80.10±0.32	83.75±0.28	73.90±0.37	71.29±0.33	73.44±0.37	-	-	-	85.27±0.20	84.65±0.23	84.44±0.17
5	**89.23±0.19**	**88.40±0.11**	**88.95±0.28**	**88.50±0.15**	**84.48±0.10**	**87.21±0.29**	**76.45±0.32**	**75.83±0.25**	**78.65±0.17**	**75.79±0.20**	**74.12±0.28**	**75.06±0.11**	**85.80±0.19**	**84.37±0.23**	**84.60±0.13**
5-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**		
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA
1	98.70±0.39	98.51±0.44	98.68±0.35	94.31±0.28	92.99±0.30	94.15±0.33	81.35±0.41	80.95±0.31	81.11±0.26	78.70±0.28	77.84±0.23	77.57±0.45	87.64±0.41	85.77±0.36	86.29±0.44
2	99.85±0.42	99.76±0.37	99.83±0.41	-	-	-	-	-	-	-	-	-	86.72±0.30	85.94±0.29	85.39±0.32
3	99.75±0.22	99.58±0.26	99.75±0.23	96.20±0.19	94.09±0.23	96.41±0.27	-	-	-	-	-	-	88.90±0.25	86.23±0.31	89.06±0.22
4	99.85±0.19	**99.83±0.25**	99.83±0.22	95.90±0.26	93.79±0.21	96.25±0.31	82.05±0.22	78.60±0.27	77.35±0.32	-	-	-	88.50±0.20	86.03±0.26	88.21±0.34
5	**99.89±0.11**	99.72±0.20	**99.81±0.29**	**96.34±0.25**	**94.17±0.18**	**95.67±0.21**	**82.63±0.25**	**79.56±0.20**	**79.40±0.31**	**80.81±0.23**	**80.39±0.18**	**81.13±0.22**	**91.60±0.27**	**90.47±0.23**	**90.35±0.19**

TABLE VI

ANALYSIS OF THE IMPACT OF DOMAIN-SPECIFIC BN LAYERS AND $\mathcal{L}_{\text{VAR}}$ IN MDFS-NET. THE RESULTS ARE SHOWN ON ONE-SHOT AND FIVE-SHOT SETTINGS FOR FIVE RS HYPERSPECTRAL DOMAINS, DEFINED IN SECTION IV-A. MDFS-NET$^{\#}$ USES l_1-NORM FOR SSL. HIGHLIGHTED IN **BOLD**

Method	Dataset														
1-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**		
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA
Without BN	68.90±0.21	68.53±0.23	67.49±0.20	65.53±0.28	63.87±0.31	65.22±0.29	68.19±0.20	68.30±0.39	67.93±0.37	48.65±0.40	48.99±0.32	47.54±0.29	73.80±0.33	72.60±0.26	72.38±0.30
Without $\mathcal{L}_{Var}$	85.37±0.23	85.14±0.19	84.80±0.24	85.00±0.31	80.56±0.26	79.41±0.20	74.27±0.30	74.37±0.36	73.09±0.22	73.90±0.39	72.44±0.31	72.29±0.30	74.18±0.44	75.51±0.37	75.27±0.46
With l_2-norm	88.60±0.30	87.32±0.39	87.44±0.30	88.13±0.29	83.70±0.38	86.60±0.34	76.03±0.25	75.50±0.31	77.82±0.25	75.30±0.25	73.79±0.26	74.95±0.23	85.61±0.34	84.19±0.30	83.44±0.21
MDFS-Net$^{\#}$	**89.23±0.19**	**88.40±0.11**	**88.95±0.28**	**88.50±0.15**	**84.48±0.10**	**87.21±0.29**	**76.45±0.32**	**75.83±0.25**	**78.65±0.17**	**75.79±0.20**	**74.12±0.28**	**75.06±0.11**	**85.80±0.19**	**84.37±0.23**	**84.60±0.13**
5-shot															
	Indian Pines			**Salinas**			**Botswana**			**KSC**			**Pavia U.**		
	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA	OA	κ	AA
Without BN	80.06±0.24	78.53±0.19	78.96±0.40	78.63±0.27	75.11±0.41	76.33±0.22	70.59±0.34	70.10±0.20	69.52±0.14	61.35±0.43	60.75±0.30	60.80±0.52	79.44±0.19	78.61±0.26	78.25±0.33
Without $\mathcal{L}_{Var}$	93.27±0.27	92.95±0.36	92.68±0.20	92.84±0.30	89.71±0.22	89.52±0.37	80.94±0.12	76.80±0.19	76.05±0.43	72.77±0.39	71.43±0.31	71.99±0.20	81.34±0.14	82.93±0.30	83.12±0.26
With l_2-norm	99.44±0.30	99.11±0.26	99.35±0.22	95.78±0.34	93.94±0.30	95.11±0.25	82.07±0.26	78.63±0.22	78.52±0.20	79.60±0.30	78.86±0.22	78.50±0.31	91.12±0.23	88.31±0.35	89.80±0.37
MDFS-Net$^{\#}$	**99.89±0.11**	**99.72±0.20**	**99.81±0.29**	**96.34±0.25**	**94.17±0.18**	**95.67±0.21**	**82.63±0.25**	**79.56±0.20**	**79.40±0.31**	**80.81±0.23**	**80.39±0.18**	**81.13±0.22**	**91.60±0.27**	**90.47±0.23**	**90.35±0.19**

More specifically, in a five-way one-shot setting, MDFS-Net significantly outperforms OA for the single domain by 3.8%, 2.9%, 2.6%, 0.8%, and 4.3% on IP, Salinas, Botswana, KSC, and Pavia U. datasets, respectively. At the same time, MDFS-Net shows improvement in OA by 1.2%, 2.1%, 1.5%, 2.6%, and 4.2% for the respected HSI datasets in the five-way five-shot setting.

3) Impact of Domain-Specific BN Layers: To study the importance of domain-specific BN layers in the MDFS-Net, we remove the domain-specific BN (i.e., BN-3D and BN-1D) layers and use the global BN layers. Due to the high diversity in the spectral and spatial properties of the different HSIs, the performance of MDFS hyperspectral classification drops considerably. We analyze that the addition of domain-specific BN layers [see Table VI (row named as without BN)] in the proposed MDFS-Net impacts retaining the domain-specific features for classification and improves the overall performance at least by 12.04%.

4) Choice of SSL for Reconstruction: In order to reconstruct the support patches from the representation space, we ablate MDFS-Net with l_1-norm and l_2-norm in the joint optimization function, defined in (8). We showcase the ablation results in Table VI (row named as with l_1 and l_2) and note that the SSL equipped with l_1-norm helps in better classifying averagely by 0.63% than l_2-norm.

C. Parameter Sensitive Analysis

1) Ablation on the Choice of α: We optimize the joint loss function defined in (8) for training our proposed MDFS-Net. To choose the best value of α, we iterate the MDFS-Net with different values of α. We show the results in Table VII and observe that as we increase the value of α ($0 \geq \alpha \geq 1$), the performance of MDFS-Net improves by a fair margin. For both one-shot and five-shot settings with $\alpha = 1$, MDFS-Net records the best scores (OA). Furthermore, we increase the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

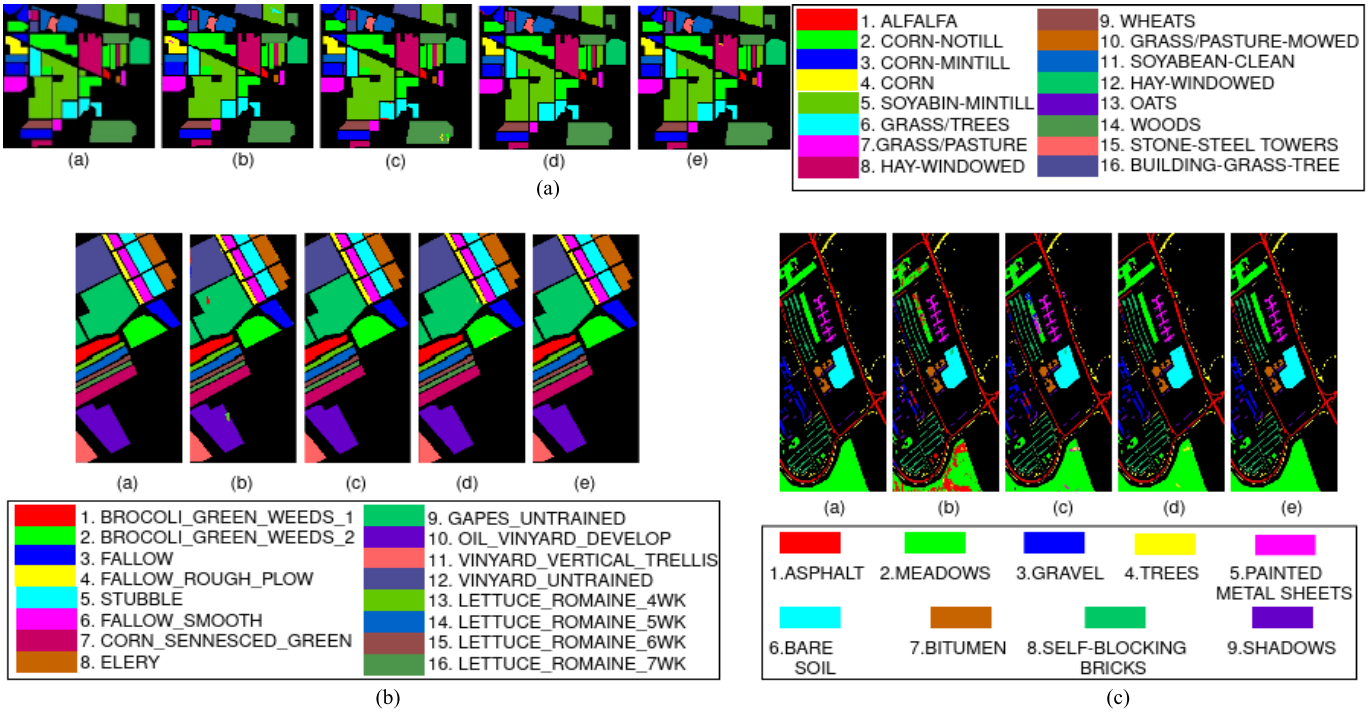


Fig. 3. Generated classification maps on (A) IP datasets, (B) Salinas, and (C) Pavia U. datasets. The subplots are (a) ground-truth, (b) single-domain one-shot, (c) multidomain one-shot, (d) single-domain five-shot, and (e) multidomain five-shot. The class labels are on the extreme right. Best viewed in color.

TABLE VII

ANALYSIS ON THE CHOICE OF α, WHICH DEFINED IN (8). WE REPORT THE PERFORMANCE OF MDFS-NET WITH OA FOR ALL THE HSI DATASETS. WE HIGHLIGHT THE BEST RESULTS IN **BOLD**

	Dataset									
	1-shot					*5-shot*				
α	**IP**	**Sal.**	**Bot.**	**KSC**	**PavU.**	**IP**	**Sal.**	**Bot.**	**KSC**	**PavU.**
	OA	OA	OA	OA	OA	OA	OA	OA	OA	OA
0.0	83.61	82.47	70.54	68.20	75.68	90.28	89.60	75.43	72.70	81.45
0.5	85.45	86.11	73.69	73.51	79.20	98.60	93.15	78.90	79.87	85.20
0.7	88.30	86.74	75.22	73.87	83.51	99.14	93.49	80.16	79.55	90.73
1.0	**89.23**	88.50	**76.45**	**75.79**	**85.80**	**99.89**	**96.34**	**82.63**	**80.81**	**91.60**
2.0	89.15	88.43	76.30	75.49	85.54	99.82	96.11	82.20	79.66	91.39
3.0	88.52	**88.65**	76.01	75.43	85.37	99.49	95.90	82.55	80.30	91.43

value of α, i.e., $\alpha > 1$, and MDFS-Net performs slightly less than with $\alpha = 1$ but beats the SOTA methods considerably.

2) Effect of $\mathcal{L}_{Var}$ in MDFS-Net: While designing the MDFS architecture for HSIs classification, we have incorporated the uncertainty reduction loss ($\mathcal{L}_{\text{Var}}$) defined in (6) for reducing the uncertainty in the feature embeddings. We ablate our model by ignoring $\mathcal{L}_{\text{Var}}$ loss from the total loss $\mathcal{L}_{\text{Total}}$ from optimization. From Table VI (row named as without $\mathcal{L}_{\text{Var}}$), we observe that $\mathcal{L}_{\text{Var}}$ helps in better MDFS classification.

D. Visualization

1) Classification Maps: In Fig. 3, we show the generated classification maps for the IP, Salinas, and Pavia U. datasets on a single domain and multidomain settings on one-shot and five-shot scenarios. We also depict the class labels for easy interpretations of the hyperspectral pixels. From the visual outputs, one can see the improvements in generation in class maps from single-domain to multidomain architectures.

We further show the comparisons of OA, k, and AA between the k-shot performance ($\mathcal{K} = 1, 5, 10$) on the Botswana and Pavia U. dataset in Fig. 4 and the MDFS-Net performance improves with increase in the number of shots.

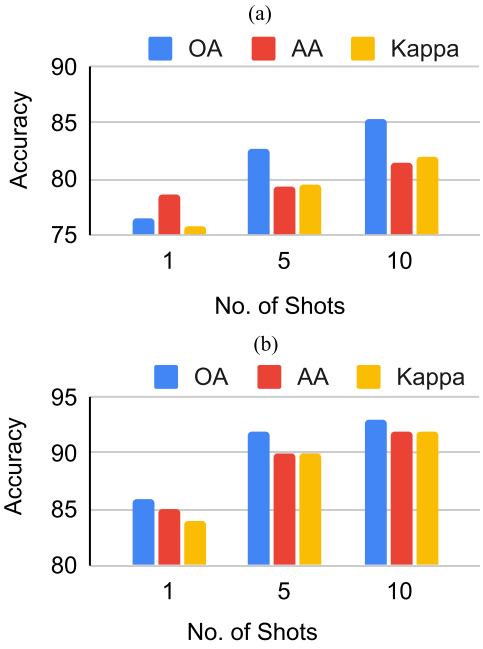


Fig. 4. Comparison between the one-shot, five-shot, and ten-shot performances of MDFS-Net on (a) Botswana and (b) Pavia U. datasets. Best view in color.

VI. CONCLUSION

We propose the task of FS-MDL in the context of HSI classification, which deals with training a shared model with multiple HSI data from nonoverlapping geographical

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

locations simultaneously. While MDFS-Net shares the domain-invariant convolution and FC layers among the domains, the domain-specific characteristics are captured in the separate batch-norm parameters. As a result, MDFS-Net can be trained conveniently on all the domains without the possibility of model overfitting or catastrophic forgetting. We showcase the performance of MDFS-Net on five benchmark HSI datasets and find that the MDL outperforms the models trained for the individual HSI data, even in the low-data regime. This establishes the information sharing of MDFS-Net, which improves the model performance. We sincerely hope that our research will open up new avenues in the future; some of the possibilities can be extending MDFS-Net for multiple modalities, different domains with domain-invariant augmentations (mix-up strategies), and incorporating the notion of open-set learning in MDFS-Net, to name a few.

REFERENCES

[1] F. Melgani and L. Bruzzone, "Support vector machines for classification of hyperspectral remote-sensing images," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, vol. 1, Jun. 2002, pp. 506–508.

[2] Y. Qian, M. Ye, and J. Zhou, "Hyperspectral image classification based on structured sparse logistic regression and three-dimensional wavelet texture features," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 4, pp. 2276–2291, Apr. 2013.

[3] S.-S. Wu, X. Qiu, E. L. Usery, and L. Wang, "Using geometrical, textural, and contextual information of land parcels for classification of detailed urban land use," *Ann. Assoc. Amer. Geographers*, vol. 99, no. 1, pp. 76–98, Jan. 2009.

[4] J. A. Benediktsson, P. H. Swain, and O. K. Ersoy, "Neural network approaches versus statistical methods in classification of multisource remote sensing data," *IEEE Trans. Geosci. Remote Sens.*, vol. 28, no. 4, pp. 540–552, Jul. 1990.

[5] J. Yue, S. Mao, and M. Li, "A deep learning framework for hyperspectral image classification using spatial pyramid pooling," *Remote Sens. Lett.*, vol. 7, no. 9, pp. 875–884, Sep. 2016.

[6] B. Liu, X. Yu, A. Yu, and G. Wan, "Deep convolutional recurrent neural network with transfer learning for hyperspectral image classification," *J. Appl. Remote Sens.*, vol. 12, no. 2, Jun. 2018, Art. no. 026028.

[7] D. Hong, X. Wu, P. Ghamisi, J. Chanussot, N. Yokoya, and X. X. Zhu, "Invariant attribute profiles: A spatial-frequency joint feature extractor for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 6, pp. 3791–3808, Jun. 2020.

[8] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[9] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "A new deep convolutional neural network for fast hyperspectral image classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 120–147, Nov. 2018.

[10] P. Chen, B. Zhang, D. Hong, Z. Chen, X. Yang, and B. Li, "FCCDN: Feature constraint network for VHR image change detection," *ISPRS J. Photogramm. Remote Sens.*, vol. 187, pp. 101–119, May 2022.

[11] X. Zhang, Y. Guo, and X. Zhang, "Hyperspectral image classification based on optimized convolutional neural networks with 3D stacked blocks," *Earth Sci. Informat.*, vol. 15, pp. 383–395, Jan. 2022.

[12] H. Xu, W. Yao, L. Cheng, and B. Li, "Multiple spectral resolution 3D convolutional neural network for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 7, p. 1248, Mar. 2021.

[13] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, Jul. 2015, Art. no. 258619.

[14] X. Wu, D. Hong, and J. Chanussot, "UIU-Net: U-Net in U-Net for infrared small object detection," *IEEE Trans. Image Process.*, vol. 32, pp. 364–376, 2023.

[15] Z. Chen et al., "Global to local: A hierarchical detection algorithm for hyperspectral image target detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5544915.

[16] D. Hong, X. Wu, J. Yao, and X. Zhu, "Beyond pixels: Learning from multimodal hyperspectral superpixels for land cover classification," *Sci. China Technol. Sci.*, vol. 65, no. 4, pp. 802–808, Apr. 2022.

[17] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.

[18] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1–10.

[19] S. Ravi and H. Larochelle, "Optimization as a model for few-shot learning," in *Proc. Int. Conf. Learn. Represent.*, 2017, pp. 1–11.

[20] Y. Chen, X. Wang, Z. Liu, H. Xu, and T. Darrell, "A new meta-baseline for few-shot learning," 2020, *arXiv:2003.04390*.

[21] J. Bai et al., "Class incremental learning with few-shots based on linear programming for hyperspectral image classification," *IEEE Trans. Cybern.*, vol. 52, no. 6, pp. 5474–5485, Jun. 2022.

[22] K. Gao, B. Liu, X. Yu, J. Qin, P. Zhang, and X. Tan, "Deep relation network for hyperspectral image few-shot classification," *Remote Sens.*, vol. 12, no. 6, p. 923, Mar. 2020.

[23] S. Liu, Q. Shi, and L. Zhang, "Few-shot hyperspectral image classification with unknown classes using multitask deep learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5085–5102, Jun. 2021.

[24] D. Pal, V. Bundele, B. Banerjee, and Y. Jeppu, "SPN: Stable prototypical network for few-shot learning-based hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[25] Z. Xue, Y. Zhou, and P. Du, "S3Net: Spectral–spatial Siamese network for few-shot hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5531219.

[26] J. Zeng, Z. Xue, L. Zhang, Q. Lan, and M. Zhang, "Multistage relation network with dual-metric for few-shot hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5510017.

[27] B. Cao, X. Ni, J.-T. Sun, G. Wang, and Q. Yang, "Distance metric learning under covariate shift," in *Proc. 22nd Int. Joint Conf. Artif. Intell.*, 2011, pp. 1204–1210.

[28] R. Mdrafi, Q. Du, A. C. Gurbuz, B. Tang, L. Ma, and N. H. Younan, "Attention-based domain adaptation using residual network for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 6424–6433, 2020.

[29] W.-G. Chang, T. You, S. Seo, S. Kwak, and B. Han, "Domain-specific batch normalization for unsupervised domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 7346–7354.

[30] K. You, M. Long, Z. Cao, J. Wang, and M. I. Jordan, "Universal domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. pattern Recognit.*, Jun. 2019, pp. 2715–2724.

[31] D. Li, Y. Yang, Y.-Z. Song, and T. M. Hospedales, "Learning to generalize: Meta-learning for domain generalization," 2017, *arXiv:1710.03463*.

[32] D. Li, Y. Yang, Y.-Z. Song, and T. M. Hospedales, "Deeper, broader and artier domain generalization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 5543–5551.

[33] K. Zhou, Z. Liu, Y. Qiao, T. Xiang, and C. C. Loy, "Domain generalization: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 4396–4415, Apr. 2023.

[34] S.-A. Rebuffi, H. Bilen, and A. Vedaldi, "Learning multiple visual domains with residual adapters," 2017, *arXiv:1705.08045*.

[35] A. Senhaji, J. Raitoharju, M. Gabbouj, and A. Iosifidis, "Not all domains are equally complex: Adaptive multi-domain learning," in *Proc. 25th Int. Conf. Pattern Recognit. (ICPR)*, Jan. 2021, pp. 8663–8670.

[36] Y. Li and N. Vasconcelos, "Efficient multi-domain learning by covariance normalization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5419–5428.

[37] B. Zhao, Y. Zhong, and L. Zhang, "A spectral–structural bag-of-features scene classifier for very high spatial resolution remote sensing imagery," *ISPRS J. Photogramm. Remote Sens.*, vol. 116, pp. 73–85, Jun. 2016.

[38] C. Zhang et al., "Joint deep learning for land cover and land use classification," *Remote Sens. Environ.*, vol. 221, pp. 173–187, Feb. 2019.

[39] J. C. Harsanyi and C.-I. Chang, "Hyperspectral image classification and dimensionality reduction: An orthogonal subspace projection approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 32, no. 4, pp. 779–785, Jul. 1994.

[40] H. Gholizadeh, J. A. Gamon, A. I. Zygielbaum, R. Wang, A. K. Schweiger, and J. Cavender-Bares, "Remote sensing of biodiversity: Soil correction and data dimension reduction methods improve assessment of α-diversity (species richness) in Prairie ecosystems," *Remote Sens. Environ.*, vol. 206, pp. 240–253, Mar. 2018.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.

[41] C. Rodarmel and J. Shan, "Principal component analysis for hyperspectral image classification," *Surv. Land Inf. Syst.*, vol. 62, no. 2, pp. 115–122, Jun. 2002.

[42] Y. Ma, C. Li, H. Li, X. Mei, and J. Ma, "Hyperspectral image classification with discriminative kernel collaborative representation and Tikhonov regularization," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 4, pp. 587–591, Apr. 2018.

[43] K. Makantasis, K. Karantzalos, A. Doulamis, and N. Doulamis, "Deep supervised learning for hyperspectral data classification through convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2015, pp. 4959–4962.

[44] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral–spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, Jun. 2015, doi: 10.1080/2150704X.2015.1047045.

[45] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3232–3245, May 2020.

[46] R. Hang, Z. Li, Q. Liu, P. Ghamisi, and S. S. Bhattacharyya, "Hyperspectral image classification with attention-aided CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2281–2293, Mar. 2021.

[47] J. M. Haut, M. E. Paoletti, J. Plaza, A. Plaza, and J. Li, "Visual attention-driven hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 8065–8080, Oct. 2019.

[48] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518615.

[49] L. Dang, L. Weng, W. Dong, S. Li, and Y. Hou, "Spectral–spatial attention transformer with dense connection for hyperspectral image classification," *Comput. Intell. Neurosci.*, vol. 2022, May 2022, Art. no. 7071485.

[50] Y. Qing, W. Liu, L. Feng, and W. Gao, "Improved transformer net for hyperspectral image classification," *Remote Sens.*, vol. 13, no. 11, p. 2216, Jun. 2021.

[51] D. Hong, J. Yao, D. Meng, N. Yokoya, and J. Chanussot, "Decoupled-and-coupled networks: Self-supervised hyperspectral image super-resolution with subpixel fusion," 2022, *arXiv:2205.03742*.

[52] D. Hong et al., "Endmember-guided unmixing network (EGU-Net): A general deep learning framework for self-supervised hyperspectral unmixing," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 11, pp. 6518–6531, Nov. 2022.

[53] X. Wu, D. Hong, and D. Zhao, "Hyper-embedder: Learning a deep embedder for self-supervised hyperspectral dimensionality reduction," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[54] Z. Han, D. Hong, L. Gao, B. Zhang, M. Huang, and J. Chanussot, "AutoNAS: Automatic neural architecture search for hyperspectral unmixing," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5532214.

[55] J. Yue, L. Fang, H. Rahmani, and P. Ghamisi, "Self-supervised learning with adaptive distillation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501813.

[56] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1199–1208.

[57] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," 2016, *arXiv:1606.04080*.

[58] M. Ren et al., "Meta-learning for semi-supervised few-shot classification," 2018, *arXiv:1803.00676*.

[59] G. Koch, R. Zemel, and R. Salakhutdinov, "Siamese neural networks for one-shot image recognition," in *Proc. ICML Deep Learn. Workshop*, vol. 2, Lille, France, 2015, pp. 1–8.

[60] B. Liu, X. Yu, A. Yu, P. Zhang, G. Wan, and R. Wang, "Deep few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2290–2304, Apr. 2019.

[61] N. A. A. Braham, L. Mou, J. Chanussot, J. Mairal, and X. X. Zhu, "Self supervised learning for few shot hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2022, pp. 267–270.

[62] J. Bai et al., "Few-shot hyperspectral image classification based on adaptive subspaces and feature transformation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5523917.

[63] P. Liu, H. Zhang, and K. B. Eom, "Active deep learning for classification of hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 10, no. 2, pp. 712–724, Feb. 2017.

[64] M. Xu, Y. Zhao, Y. Liang, and X. Ma, "Hyperspectral image classification based on class-incremental learning with knowledge distillation," *Remote Sens.*, vol. 14, no. 11, p. 2556, May 2022.

[65] E. Tzeng, J. Hoffman, T. Darrell, and K. Saenko, "Simultaneous deep transfer across domains and tasks," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2015, pp. 4068–4076.

[66] H. Wang, Y. Cheng, C. L. P. Chen, and X. Wang, "Hyperspectral image classification based on domain adversarial broad adaptation network," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5517813.

[67] X. Wang, Y. Li, and Y. Cheng, "Hyperspectral image classification based on unsupervised heterogeneous domain adaptation CycleGan," *Chin. J. Electron.*, vol. 29, no. 4, pp. 608–614, Jul. 2020.

[68] P. Z. Ramirez, A. Tonioni, S. Salti, and L. Di Stefano, "Learning across tasks and domains," 2019, *arXiv:1904.04744*.

[69] S. Rakshit, A. Mohanty, R. Chavhan, B. Banerjee, G. Roig, and S. Chaudhuri, "FRIDA—Generative feature replay for incremental domain adaptation," *Comput. Vis. Image Understand.*, vol. 217, Mar. 2022, Art. no. 103367.

[70] A. Rosenfeld and J. K. Tsotsos, "Incremental learning through deep adaptation," 2017, *arXiv:1705.04228*.

[71] A. Senhaji, J. Raitoharju, M. Gabbouj, and A. Iosifidis, "Not all domains are equally complex: Adaptive multi-domain learning," 2020, *arXiv:2003.11504*.

[72] H. Bilen and A. Vedaldi, "Universal representations: The missing link between faces, text, planktons, and cat breeds," 2017, *arXiv:1701.07275*.

[73] S.-A. Rebuffi, H. Bilen, and A. Vedaldi, "Efficient parametrization of multi-domain deep neural networks," 2018, *arXiv:1803.10082*.

[74] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

[75] A. Paszke et al., "Automatic differentiation in PyTorch," in *Proc. 31st Conf. Neural Inf. Process. Syst. (NIPS-W)*, 2017, pp. 1–4.



Ankit Jha received the B.Tech. degree in electronics and communication engineering from Rajasthan Technical University, Kota, India, in 2015, and the M.Tech. degree in modeling and simulation from the Defence Institute of Advanced Technology, Pune, India, in 2018. He is currently pursuing the Ph.D. degree in machine learning and visual computing for remote sensing with the Centre of Studies in Resources Engineering (CSRE), IIT Bombay, Mumbai, India.

He has collaborative work with the University of Trento, Trento, Italy, and the Mohamed bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates. His research interests include multitask learning, multimodal learning, domain generalization with prompt learning, multidomain learning, and few-shot learning.



Biplab Banerjee (Senior Member, IEEE) received the bachelor's degree from the West Bengal University of Technology, Kolkata, India, in 2008, the M.E. degree in computer science and engineering from Jadavpur University, Kolkata, India, in 2010, and the Ph.D. degree in image analysis from IIT Bombay, Mumbai, India, in 2015.

He was a Post-Doctoral Researcher with the University of Caen Basse-Normandy, Caen, France, and the Istituto Italiano di Technologia Genova, Genoa, Italy. He was an Assistant Professor with the Department of Computer Science and Engineering, IIT Roorkee, Roorkee, India, from 2016 to 2018. He holds a Visiting Professor Tenure at Technische Universität München, Munich, Germany; Ghent University, Ghent, Belgium; and Kyungpook National University, Daegu, South Korea, to name a few. He is currently an Associate Professor with the Centre of Studies in Resources Engineering (CSRE) and the Center of Machine Intelligence and Data Science (MInDS), IIT Bombay, where he is closely associated with the Vision and Image Processing Group, Department of Electrical Engineering. Besides, he is also an Engineering Advisor of AI at AWL Inc., Tokyo, Japan. His research interests include zero-shot learning, meta-learning, multitask learning, domain adaptation and transfer learning, multimodal analysis of remote sensing data, and deep reinforcement learning.

Dr. Banerjee was a recipient of the Excellence in Ph.D. Thesis Award for his Ph.D. thesis from IIT Bombay.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 09:52:59 UTC from IEEE Xplore. Restrictions apply.